

“board_demo” Analysis

This section aims to demonstrate the control principle and program of “ros_robot_controller_sdk.py” in the folder “board_demo”.

1. “ros_robot_controller_sdk.py” Control Principle

The “ros_robot_controller_sdk.py” is a Python SDK designed for serial communication between the Raspberry Pi and STM32 development board. It enables the control of various hardware components such as LEDs, buzzers, and motors. The Raspberry Pi 5B communicates with the STM32 via serial port using the Cyclic Redundancy Check (CRC) algorithm to drive the underlying hardware on the MM32 chip. Let's take a brief analysis in the following section:

```
12 class PacketControllerState(enum.IntEnum):
13     # The format of the communication protocol
14     # 0xAA 0x55 Length Function ID Data Checksum
15     PACKET_CONTROLLER_STATE_STARTBYTE1 = 0
16     PACKET_CONTROLLER_STATE_STARTBYTE2 = 1
17     PACKET_CONTROLLER_STATE_LENGTH = 2
18     PACKET_CONTROLLER_STATE_FUNCTION = 3
19     PACKET_CONTROLLER_STATE_ID = 4
20     PACKET_CONTROLLER_STATE_DATA = 5
21     PACKET_CONTROLLER_STATE_CHECKSUM = 6
22
23 class PacketFunction(enum.IntEnum):
24     # Control functions that can be achieved through the serial port
25     PACKET_FUNC_SYS = 0
26     PACKET_FUNC_LED = 1 # LED control
27     PACKET_FUNC_BUZZER = 2 # Buzzer control
28     PACKET_FUNC_MOTOR = 3 # Motor control
29     PACKET_FUNC_PWM_SERVO = 4 # PWM servo control, 1-4 on the board from inside to outside
30     PACKET_FUNC_BUS_SERVO = 5 # Bus servo control
31     PACKET_FUNC_KEY = 6 # button to get
32     PACKET_FUNC_IMU = 7 # IMU acquisition
33     PACKET_FUNC_GAMEPAD = 8 # Handle acquisition
34     PACKET_FUNC_SBUS = 9 # Model airplane remote control
35     PACKET_FUNC_OLED = 10 # OLED displays content settings
36     PACKET_FUNC_RGB = 11 # Set the RGB color
37     PACKET_FUNC_NONE = 12
```

① The data packet format is “0xAA 0x55 Length Function ID Data Checksum”. Among them, “0xAA” and “0x55” are start bytes used to identify the start of the data packet; “Length” indicates its length; “Function” indicates the type of function; “ID” represents the device ID; “Data” is the content of the data; and “Checksum” is the checksum.

② Data packet parsing: The received data packet is parsed based on the

different “Function” fields. The SDK uses a state machine to parse the data packet, and each field of the data packet is parsed step by step based on the current state.

③ Function types: The “Function” field of the data packet represents different types of functions. The SDK defines multiple enumeration types, such as “PacketFunction” represents the different types of functions; “PACKET_FUNC_LED” represents LED control; “PACKET_FUNC_BUZZER” represents buzzer control; and “PACKET_FUNC_PWM_SERVO” represents PWM servo control.

```

69 def checksum_crc8(data):
70     # verify
71     check = 0
72     for b in data:
73         check = crc8_table[check ^ b]
74     return check & 0x00FF
75
76 class SBusStatus:
77     def __init__(self):
78         self.channels = [0] * 16;
79         self.channel_17 = False
80         self.channel_18 = False
81         self.signal_loss = True
82         self.fail_safe = False
83
84 class Board:
85     buttons_map = {
86         'GAMEPAD_BUTTON_MASK_L2': 0x0001,
87         'GAMEPAD_BUTTON_MASK_R2': 0x0002,
88         'GAMEPAD_BUTTON_MASK_SELECT': 0x0004,
89         'GAMEPAD_BUTTON_MASK_START': 0x0008,
90         'GAMEPAD_BUTTON_MASK_L3': 0x0020,
91         'GAMEPAD_BUTTON_MASK_R3': 0x0040,
92         'GAMEPAD_BUTTON_MASK_CROSS': 0x0100,
93         'GAMEPAD_BUTTON_MASK_CIRCLE': 0x0200,
94         'GAMEPAD_BUTTON_MASK_SQUARE': 0x0800,
95         'GAMEPAD_BUTTON_MASK_TRIANGLE': 0x1000,
96         'GAMEPAD_BUTTON_MASK_L1': 0x4000,
97         'GAMEPAD_BUTTON_MASK_R1': 0x8000
98     }
99
100 def __init__(self, device="/dev/ttyAMA0", baudrate=1000000, timeout=5):
101     self.enable_recv = False
102     self.frame = []
103     self.recv_count = 0
104
105     self.port = serial.Serial(None, baudrate, timeout=timeout)
106     self.port.rts = False
107     self.port.dtr = False
108     self.port.setPort(device)
109     self.port.open()
110
111     self.state = PacketControllerState.PACKET_CONTROLLER_STATE_STARTBYTE1
112     self.servo_read_lock = threading.Lock()
113     self.pwm_servo_read_lock = threading.Lock()
114

```

① The “checksum_crc8(data)” function calculates the CRC8 checksum. It receives a data list as input, performs XOR and table look-ups on the data to obtain the checksum, and returns the value of the checksum.

② Initialize serial port through the specified device, baud rate, and timeout. Queues are set up for different types of data: system, bus servo, PWM servo, keys, IMU, joystick, and SBUS.

- ③ A separate thread is started simultaneously to receive data from the board “recv_task”.
- ④ The class defines various methods (packet_report_XXX) to parse different types of report data received from the board. The parsing methods process the received data and place it in the corresponding queue for further processing.
- ⑤ The class provides the method to send commands to the board for different functions, such as the control of the LED, buzzer, and motor.
- ⑥ The “recv_task” method runs in an independent thread and continuously reads data from the serial port.
- ⑦ It processes the received bytes, constructs data packets based on the communication protocol, then uses the appropriate parsing method to parse the data based on the function types of the data packets, and places the parsed data into the corresponding queue. The queue is called then for further processing.

2. “board_demo” File

Program	Function
bus_servo_speed.py	Control the rotation speed of the bus servo.
bus_servo_turn.py	Control the rotation of the bus servo.
bus_servos_read.py	Read the data of bus servo.
buzzer_control_demo	Control the buzzer.

gpio_control	GPIO application.
key.py	Keys control.
motor_control_demo.py	Control DC motor.
pwm_servo_control_demo.py	Control multiple the PWM servos.
pwm_servo_speed.py	Control the speed of the PWM servo.
rgb_control_demo.py	Control RGB light.
rgb_flashing.py	Control RGB light flashing.
ros_robot_controller_sdk.py	Control the underlying file of the expansion board.
single_servos.py	Control the rotation of PWM servo.